

# CSE314 – Compiler Design Lab

## Practice Problem Set

Instructor: S.K.M Shadikul Islam

---

### Section A: Lexical Analysis with Flex (Weeks 1–5)

1. **Whitespace & Comment Stripper** — Write a Flex program that reads a C-like source file and outputs it with all whitespace (extra spaces, tabs, newlines) collapsed to a single space, and all `//` and `/* */` comments removed.
2. **Word & Line Counter** — Write a Flex program that counts the number of words, lines, and characters in a text file (like a simplified `wc`).
3. **Identifier & Keyword Classifier** — Write a lexical analyzer that scans a C-like file and classifies each token as `KEYWORD`, `IDENTIFIER`, `NUMBER`, `OPERATOR`, or `PUNCTUATION`, printing each token with its class.
4. **Identifier Length Validator** — Extend Problem 3 so identifiers longer than a fixed length (e.g., 31 characters) are flagged as errors, while still being recognized as identifiers.
5. **Number Classifier** — Write a Flex program that distinguishes integers, floating-point numbers (with optional exponent, e.g., `3.14e-2`), and invalid number formats (e.g., `3.14.5`), reporting each with its type or an error message.
6. **String & Character Literal Scanner** — Write a Flex program that correctly recognizes string literals (`" . . . "`) and character literals (`' . . . '`), including escape sequences like `\n`, `\t`, `\"`, and reports unterminated strings as lexical errors.
7. **Regex Pattern Recognizer** — Write a Flex program that recognizes strings matching each of the following regular expressions over `{a, b}`, and labels which pattern (if any) a given input string matches: `a*`, `a*b+`, `(ab)*`, `abb`.
8. **Operator Validator** — Write a Flex program that validates C-style operators (`+`, `-`, `*`, `/`, `==`, `!=`, `<=`, `>=`, `&&`, `||`, `++`, `--`, etc.), rejecting malformed combinations like `++*` or `=+`.
9. **Line & Column Tracker** — Modify any earlier lexer so that every token printed also reports its line number and column position in the source file (useful groundwork for error reporting later).
10. **Simple Preprocessor** — Write a Flex program that expands `#define NAME VALUE` macros textually within a source file (a simplified `#define` handler, ignoring function-like macros).

## Section B: Finite Automata Theory (Weeks 8–9)

11. **Thompson's Construction (NFA)** — Given a regular expression as input, construct the equivalent NFA and print its transition table.
12. **NFA to DFA (Subset Construction)** — Given the transition table of an NFA, implement the subset construction algorithm to output the equivalent DFA's transition table.
13. **DFA Minimization** — Given a DFA transition table, implement an algorithm to minimize the number of states (partition refinement / table-filling method).
14. **DFA-based String Matcher** — Given a DFA transition table and a set of input strings, write a program that determines whether each string is accepted or rejected.
15. **Regex-to-DFA Direct Construction** — Given a simple regular expression (e.g., `(a|b)*abb`), directly build and simulate its DFA to accept/reject test strings, without going through the NFA step explicitly.

## Section C: Symbol Table (Week 6)

16. **Basic Symbol Table** — Implement a symbol table (using a hash table or linked list) that stores identifier name, data type, scope, and line of first declaration, supporting `insert` and `lookup` operations.
17. **Scoped Symbol Table** — Extend Problem 16 to support nested scopes (e.g., global vs. function-local), so that `lookup` searches the innermost scope outward, and redeclaration within the same scope is flagged as an error.
18. **Symbol Table + Lexer Integration** — Integrate a Flex-based lexer with your symbol table so that every identifier encountered during scanning is automatically inserted or looked up, and duplicate declarations are reported.

## Section D: Parsing with Bison (Weeks 10, 13)

19. **Arithmetic Expression Evaluator** — Write a Bison grammar (with a Flex lexer) that evaluates arithmetic expressions with `+`, `-`, `*`, `/`, parentheses, and correct operator precedence.
20. **Syntax Checker for Declarations** — Write a Bison grammar that accepts valid C-style variable declaration statements (e.g., `int x, y; float z = 3.5;`) and rejects malformed ones, reporting the line of the error.
21. **Predictive (LL(1)) Parser for a Mini-Language** — Given a small grammar (e.g., for if-else and assignment statements), manually construct the FIRST/FOLLOW sets and predictive parsing table, then implement a table-driven predictive parser in C.
22. **Shift-Reduce Parser Trace** — Given a simple grammar and an input string, manually trace (and then implement) a shift-reduce parser that shows the stack contents and action (shift/reduce) at every step.
23. **Bison-based Calculator with Variables** — Extend Problem 19 so the language supports variable assignment (`x = 5; y = x + 3;`) using your symbol table from Section C.
24. **Syntax Error Recovery** — Modify any Bison grammar from above to implement simple panic-mode error recovery (using Bison's `error` token) so the parser reports an error and continues instead of stopping at the first one.

## Section E: Integration / Mini-Project Practice (Weeks 7, 11, 12)

25. **Directed Acyclic Graph (DAG) Builder** — Given a sequence of arithmetic assignment statements (e.g., `a = b + c; d = a - b;`), build the DAG representation showing shared sub-expressions, and print the DAG structure or common subexpressions detected.
26. **Mini Compiler Front-End (Capstone Practice)** — Combine a Flex lexer, Bison parser, and symbol table to process a small language of your choosing (arithmetic + variable declarations + if/else), reporting lexical, syntax, and basic semantic (undeclared variable) errors — a dry run for the final lab project.